

Projet de fin de module — BarterSwap

API d'échange de compétences entre particuliers

Présentation du concept

BarterSwap est une plateforme qui permet à des particuliers d'échanger leurs compétences sans transaction monétaire. Le système fonctionne avec un **crédit-temps** : chaque heure de service rendue donne droit à une heure de service reçue.

Ce n'est pas :

- Une plateforme de freelance (pas d'argent)
- Une API de tutorat (pas limité à l'éducation)
- Un réseau social (pas de fil d'actualité, pas de likes)
- Un système de troc direct (les échanges sont différés via les crédits)

C'est une **banque de temps** : le temps est la monnaie d'échange.

Objectifs pédagogiques

Le projet permet de mettre en pratique l'ensemble des concepts abordés dans le cours :

- Séparation des responsabilités dans le code (logique métier, exposition HTTP, stockage)
- API REST complète avec la stdlib (`net/http` , `encoding/json`)
- Gestion d'erreurs idiomatique (sentinelles, wrapping, `errors.Is / As`)
- Middlewares (logging, recovery, CORS, auth basique)
- Tests unitaires et tests d'API avec `httptest`
- Package `context` pour les timeouts et l'annulation
- Package `database/sql` pour l'accès à la base de données relationnelle

Contraintes techniques

- **Langage** : Go uniquement
 - **Base de données** : PostgreSQL, MySQL ou MariaDB au choix
 - **Driver base de données** : le pilote approprié est autorisé (`github.com/lib/pq`, `github.com/go-sql-driver/mysql`, etc.) — c'est la seule dépendance externe autorisée
 - **Pas d'ORM** : utilisation du package `database/sql` de la `stdlib` uniquement (pas de GORM, Ent, etc.)
 - **Structure** : un seul package Go (pas de sous-packages internes)
 - **Pas de mutex** : pas de `sync.Mutex` ni `sync.RWMutex` (la base de données gère la concurrence)
 - **Pas de framework externe** : ni Gin, ni Echo, ni Chi
 - **Pas de système d'authentification avancé** : un simple header `X-User-ID` suffit
-

Contenu fonctionnel

1. Gestion des utilisateurs

Méthode	Path	Description
POST	<code>/api/users</code>	Créer un compte (crédits de bienvenue attribués automatiquement)
GET	<code>/api/users/{id}</code>	Profil public d'un utilisateur
PUT	<code>/api/users/{id}</code>	Modifier son profil
GET	<code>/api/users/{id}/skills</code>	Compétences d'un utilisateur
PUT	<code>/api/users/{id}/skills</code>	Définir ses compétences

Structure utilisateur :

```
type User struct {  
    ID          int          `json:"id"`
```

```

Pseudo      string    `json:"pseudo"`
Bio          string    `json:"bio,omitempty"`
Ville        string    `json:"ville,omitempty"`
Skills       []Skill   `json:"skills,omitempty"`
CreditBalance int       `json:"credit_balance"` // crédits-
temps disponibles
CreatedAt    string    `json:"created_at"`
}

```

Structure compétence :

```

type Skill struct {
    Nom          string `json:"nom"`           // ex: "Jardinage"
    Niveau       string `json:"niveau"`       // "débutant",
    "intermédiaire", "expert"
}

```

Note

Un utilisateur peut proposer plusieurs compétences. Les `skills` sont écrasées à chaque PUT (pas d'ajout individuel).

Tip

À la création d'un compte, **10 crédits de bienvenue** sont automatiquement attribués à l'utilisateur. Cela lui permet de pouvoir échanger ses premiers services avant même d'en avoir rendu.

2. Gestion des annonces de services

Un utilisateur publie une annonce pour proposer un service lié à une de ses compétences.

Méthode	Path	Description
GET	/api/services	Liste des services (avec filtres optionnels)
POST	/api/services	Créer une annonce de service
GET	/api/services/{id}	Détail d'un service
PUT	/api/services/{id}	Modifier son annonce
DELETE	/api/services/{id}	Supprimer son annonce
GET	/api/services?categorie={cat}	Filtrer par catégorie
GET	/api/services?ville={ville}	Filtrer par ville
GET	/api/services?search={mot-clé}	Recherche textuelle

Structure service :

```

type Service struct {
    ID            int           `json:"id"`
    ProviderID    int           `json:"provider_id"`
    Titre         string        `json:"titre"`
    Description    string        `json:"description,omitempty"`
    Categorie     string        `json:"categorie"`
    DureeMinutes  int           `json:"duree_minutes"` // durée
estimée
    Credits       int           `json:"credits"` // coût
en crédits-temps
    Ville         string        `json:"ville,omitempty"`
    Actif         bool          `json:"actif"`
    CreatedAt     string        `json:"created_at"`
}

```

Catégories d'échange (liste fermée) :

Informatique, Jardinage, Bricolage, Cuisine, Musique, Langues, Sport, Tutorat, Déménagement, Photographie, Animalier, Couture, Autre

⚠ Warning

Le filtrage et la recherche doivent être implémentés **côté serveur** (pas de filtrage client). Utilisez les query parameters de l'URL.

3. Système d'échange (réservation)

Le cœur du projet : gérer les demandes d'échange entre utilisateurs.

Méthode	Path	Description
POST	/api/exchanges	Créer une demande d'échange
GET	/api/exchanges	Liste des échanges (requêtes + reçus)
GET	/api/exchanges/{id}	Détail d'un échange
PUT	/api/exchanges/{id}/accept	Accepter une demande
PUT	/api/exchanges/{id}/reject	Refuser une demande
PUT	/api/exchanges/{id}/complete	Marquer comme terminé
PUT	/api/exchanges/{id}/cancel	Annuler (demandeur ou offreur)
GET	/api/exchanges?status={status}	Filtrer par statut

Cycle de vie d'un échange :

```
pending → accepted → completed
    ↓           ↓
rejected  cancelled
```

Règles métier importantes :

1. Un utilisateur ne peut pas s'échanger un service à lui-même
2. Un service ne peut avoir qu'un seul échange en statut `pending` ou `accepted` à la fois

3. Quand un échange passe en `accepted` :
 - Les crédits sont **bloqués** (déduits du solde du demandeur, mais pas encore crédités à l'offreur)
4. Quand un échange passe en `completed` :
 - Les crédits sont définitivement transférés à l'offreur
5. Quand un échange est `cancelled` ou `rejected` :
 - Les crédits bloqués sont **restitués** au demandeur
6. Un utilisateur ne peut pas lancer d'échange s'il n'a pas assez de crédits

Tip

Implémentez le système de crédits comme un **journal de transactions** plutôt qu'un simple solde. Cela permet de tracer chaque opération et de détecter les incohérences.

Structure d'échange :

```
type Exchange struct {
    ID            int    `json:"id"`
    ServiceID     int    `json:"service_id"`
    RequesterID   int    `json:"requester_id"` // celui qui
demande
    OwnerID       int    `json:"owner_id"`     // celui
qui propose
    Status        string `json:"status"`       // pending,
accepted, rejected, cancelled, completed
    CreatedAt     string `json:"created_at"`
    UpdatedAt     string `json:"updated_at"`
}
```

Structure transaction crédit :

```
type CreditTransaction struct {
    ID            int    `json:"id"`
    UserID        int    `json:"user_id"`
    ExchangeID    int    `json:"exchange_id"`
    Montant       int    `json:"montant"`     // positif =
```

```
crédit, négatif = débit
    Type          string `json:"type"`           // "earn",
    "spend", "refund"
    CreatedAt     string `json:"created_at"`
}
```

4. Évaluations

Après qu'un échange est `completed`, les deux parties peuvent s'évaluer mutuellement.

Méthode	Path	Description
POST	/api/exchanges/{id}/review	Donner un avis sur un échange terminé
GET	/api/users/{id}/reviews	Avis reçus par un utilisateur
GET	/api/services/{id}/reviews	Avis sur un service

Règles :

- Un utilisateur ne peut laisser qu'un seul avis par échange
- L'avis ne peut être modifié ni supprimé
- Note de 1 à 5

Structure :

```
type Review struct {
    ID          int    `json:"id"`
    ExchangeID  int    `json:"exchange_id"`
    AuthorID    int    `json:"author_id"`
    TargetID    int    `json:"target_id"`
    Note        int    `json:"note"`           // 1-5
    Commentaire string `json:"commentaire,omitempty"`
    CreatedAt   string `json:"created_at"`
}
```

5. Tableau de bord / Statistiques

Méthode	Path	Description
GET	/api/users/{id}/stats	Statistiques d'un utilisateur

Données retournées :

```
type UserStats struct {
    UserID          int    `json:"user_id"`
    ServicesActifs  int    `json:"services_actifs"`
    EchangesCompletes int  `json:"echanges_completes"`
    CreditBalance   int    `json:"credit_balance"`
    NoteMoyenne     float64 `json:"note_moyenne"`
    NbAvis          int    `json:"nb_avis"`
    TotalGagne      int    `json:"total_gagne" //
    crédits gagnés au total
    TotalDepense    int    `json:"total_depense" //
    crédits dépensés au total
}
```

Architecture

Organisez votre code de manière propre et lisible comme vu en cours : séparez clairement les responsabilités au sein de votre unique package. La logique métier (règles de gestion, validations, enchaînements) ne doit pas être mélangée avec le code d'exposition HTTP ou le code de stockage.

Caution

Les fonctions qui traitent les requêtes HTTP ne doivent **pas** contenir de logique métier. Toute règle de gestion (vérification de solde, conflit de réservation, cycle de vie d'un échange) doit être isolée dans des fonctions dédiées.

Critères d'évaluation (soutenance 3h)

Note sur 20 (dont 5 points bonus à la discrétion du jury)

Critère	Points	Détail
Fonctionnalités	/5	Tous les endpoints fonctionnent, règles métier respectées
Architecture	/3	Séparation des couches, organisation du code, nommage
Tests	/3	Couverture $\geq 60\%$, tests table-driven, tests d'API
Qualité du code	/2	gofmt , go vet , pas de warnings, idiomatique Go
Documentation	/1	README clair, commentaires godoc , exemples curl
Gestion d'erreurs	/1	Messages d'erreur cohérents, codes HTTP appropriés
Bonus (jury)	/5	Coup de cœur, originalité, dépassement du sujet
Total	/20	

Exemples de cas métier à tester (non exhaustif)

1. Créer un utilisateur → succès 201
2. Créer un utilisateur avec pseudo vide → 400
3. Publier un service avec une compétence que l'utilisateur n'a pas → 400
4. Demander un échange sur son propre service → 400
5. Demander un échange sans crédits suffisants → 400
6. Demander un échange sur un service déjà réservé → 409 (conflit)
7. Accepter un échange → crédits bloqués, statut à "accepted"
8. Compléter un échange → crédits transférés, statut à "completed"
9. Annuler un échange → crédits restitués
10. Noter un échange non terminé → 400

11. Noter deux fois le même échange → 400
12. Récupérer les stats → toutes les valeurs cohérentes

README attendu

Le fichier `README.md` à la racine doit contenir :

```
# BarterSwap – API d'échange de compétences
```

```
## Installation
```

```
git clone <url>
cd barterswap
go mod tidy
go run .
```

Endpoints

(Tableau récapitulatif de tous les endpoints)

Exemples d'utilisation

(3-4 exemples complets avec curl)

Tests

```
go test -v -cover ./...
```

Rendu

- **Code** : dépôt Git complet avec historique (dossier `.git` inclus)
- **Soutenance** : 3h (2 modules) pour 10 groupes — **10 minutes par groupe**
 - 6 min de démonstration (en direct, avec curl, cas nominaux + cas d'erreur, présentation de l'architecture intégrée au fil de la démo)

- 4 min de questions / test de résilience
- **Groupe** : 3 personnes obligatoirement (sauf dérogation exceptionnelle)